



Security Audit Report

Hyvechain precompiles

v1.0

May 15, 2026

Table of Contents

Table of Contents	2
License	5
Disclaimer	6
Introduction	7
Purpose of This Report	7
Codebase Submitted for the Audit	8
Methodology	9
Functionality Overview	9
How to Read This Report	10
Code Quality Criteria	11
Summary of Findings	12
Detailed Findings	16
1. TWAP records are never pruned, allowing chain halt via submit-then-cancel spam	16
2. Unshield validation does not bind amounts or recipients to the ZK proof, allowing full pool drainage	17
3. SubmitOrder accepts arbitrary orders without verifying the caller's solvency or escrowing tokens	17
4. Stealth address generation uses broken cryptography	18
5. Shield path produces no shielded note, leaving the privacy module structurally non-functional	19
6. Convert anchor is unbound, allowing a prover to mint shielded staking rewards and airdrop tokens at arbitrary multipliers	19
7. DarkPool precompile does not populate order.Commitment, causing all of a user's orders to share one nullifier	20
8. ExpireRFQs scans every RFQ every block	21
9. DarkPool keeper events leak plaintext order, RFQ, and match details	22
10. SubmitIcebergOrder and SubmitTWAPOrder emit the hidden total amount, defeating the privacy feature	22
11. Settlement verifier output is always zero because HexToAddress is applied to bech32	23
12. getSettlementBatch returns JSON bytes that Solidity cannot decode	23
13. MASP Shield ignores ShieldedRecipient, crediting the sender instead of the payment address	24
14. Precompile ABI definitions mismatch handlers, rendering darkpool and stealth non-functional	25
15. getSettlement passes incorrect types, causing runtime panics	25
16. Darkpool entity ID generation diverges between EVM and SDK paths	26
17. announce ABI omits fields required by MsgAnnounce	27
18. Unshield destination fields are not cryptographically bound to the masp transaction, enabling front-running	28

19. Per-asset value balance collapsed into a scalar, allowing cross-asset minting	29
20. FFI computation cost is not gas-metered, enabling cheap denial of service of the validator wall-clock	30
21. Precompile error path bypasses EVM gas accounting and lacks the upstream RunAtomic patch	30
22. Darkpool order IDs collide for identical orders within the same block	31
23. Missing validations in darkpool and stealth precompiles	32
24. MASP precompile does not emit EVM events for state-changing operations	33
25. registerMetaAddress accepts unvalidated stealth pubkeys	33
26. RFQ user-specified maxPrice is discarded	34
27. RFQ's EVM ExpiryBlock is hardcoded	34
28. Settlement not-found returns zero struct instead of reverting	35
29. getSettlementBatch accepts an unbounded input length	35
30. getAnnouncementsSince always scans the full announcement store	35
31. Genesis import drops the Nullifiers field, allowing every previously spent note to become re-spendable after migration	36
32. Rust panics across the FFI boundary abort the validator process and bypasses Cosmos SDK panic recovery	36
33. Similar MASP proof verified up to three times per transaction	37
34. Incorrect error handling in ffi.InitParams	38
35. profile.release does not enable overflow-checks	38
36. Redundant synthetic pseudo-nullifier in unshieldSimplified shares a KV namespace with real Sapling nullifiers	39
37. Anchor verification accepts only the current Merkle root, with no rolling history	40
38. Validator-side filesystem hardening is not enforced for MASP parameter files	40
39. No denom allow-list on shield, unshield, or shielded-transfer paths allows accepting any bank denom into the MASP pool	41
40. try_decrypt_note is an explicit stub returning None, preventing wallet syncs	42
41. Synthetic simplified-unshield nullifier collides for same-block, same-recipient, same-amount withdrawals	42
42. FFI JSON wire format carries no version byte, so silent schema drift between Rust and Go corrupts verification results	43
43. Hardcoded BranchId::MASP in Transaction::read locks the chain to one BranchId without an upgrade path	44
44. Orphan FFI exports lack any Go caller, including the safety-critical hyve_masp_check_invariant	44
45. keeper.NewKeeper hardcodes DefaultParams, causing genesis-declared parameters to be silently overridden	45
46. Caller-supplied NullifierSeed is silently overwritten by the keeper	46
47. CountOpenOrdersByUser iterates the entire open-order set on every call instead of maintaining a per-user counter	46
48. hyve_masp_key_bundle_from_seed does not validate seed entropy before deriving the spending key	47
49. verify_masp_transaction accepts masp transactions with transparent legs, even	

though the chain integration ignores them	47
50. registerMetaAddress does not implement the Label field	48
51. getMatchHistory requires a pair argument but discards it	48
52. getAnnouncements returns hex-encoded ASCII instead of decoded key bytes	49
53. shieldedBalance query is pool-wide, not per-user	49
54. Sapling nullifiers and shielded balances are duplicated structurally across Rust and Go state	50
55. ShieldedPool uses HashSet and HashMap with the default RandomState hasher, producing non-deterministic serialization	50
Appendix	52
1. Command to compute test coverage	52

License



THIS WORK IS LICENSED UNDER A [CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE](https://creativecommons.org/licenses/by-nc/4.0/).

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUCT ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

<https://oaksecurity.io/>
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by HYVECHAIN LTD to perform a security audit of the Hyvechain precompiles.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/hyveblockchain-dev/HYVECHAIN
Commit	3735553de794d600ce7d32cf8fb8e7bcc8b6f129
Scope	The scope of the audit was restricted to the <code>.go</code> files in the following directory: • <code>precompiles/masp/*</code> • <code>precompiles/stealth/*</code> • <code>precompiles/settlement/*</code> • <code>precompiles/darkpool/*</code>
Fixes verified at commit	c5743f4fc931baba32186b4021be9b76d36e048a Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
 - a. Race condition analysis
 - b. Under-/overflow issues
 - c. Key management vulnerabilities
4. Report preparation

Functionality Overview

HYVECHAIN is a Cosmos SDK-based EVM chain that offers privacy-preserving DeFi via precompiles. Specifically, a shielded pool (MASP) for private token deposits/withdrawals/transfers via ZK proofs, a darkpool for anonymous order matching with TWAP and iceberg support, a settlement layer that finalizes matched trades, and stealth addresses for unlinkable payment receipts.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	Medium-High	-
Code readability and clarity	Low-Medium	-
Level of documentation	Low-Medium	Brief godoc on functions. No specs, architecture documentations, or rationale for critical design choices.
Test coverage	Low → Medium-High	Test coverage at the audited commit was 0%. The client has increased the coverage in commit <code>02f6bda</code> for the following modules: • <code>x/settlement</code>: 88.3% (166/188) • <code>x/stealth</code>: 90.1% (146/162) • <code>x/darkpool</code>: 75.7% (980/1294) • <code>x/masp</code>: 44.2% (401/908) For more information on how we compute test coverages, please refer to this section in the appendix.

Summary of Findings

No	Description	Severity	Status
1	TWAP records are never pruned, allowing chain halt via submit-then-cancel spam	Critical	Resolved
2	Unshield validation does not bind amounts or recipients to the ZK proof, allowing full pool drainage	Critical	Resolved
3	SubmitOrder accepts arbitrary orders without verifying the caller's solvency or escrowing tokens	Critical	Resolved
4	Stealth address generation uses broken cryptography	Critical	Resolved
5	Shield path produces no shielded note, leaving the privacy module structurally non-functional	Critical	Resolved
6	Convert anchor is unbound, allowing a prover to mint shielded staking rewards and airdrop tokens at arbitrary multipliers	Critical	Resolved
7	DarkPool precompile does not populate order.Commitment, causing all of a user's orders to share one nullifier	Major	Resolved
8	ExpireRFQs scans every RFQ every block	Major	Resolved
9	DarkPool keeper events leak plaintext order, RFQ, and match details	Major	Resolved
10	SubmitIcebergOrder and SubmitTWAPOrder emit the hidden total amount, defeating the privacy feature	Major	Resolved
11	Settlement verifier output is always zero because HexToAddress is applied to bech32	Major	Resolved
12	getSettlementBatch returns JSON bytes that Solidity cannot decode	Major	Resolved
13	MASP Shield ignores ShieldedRecipient, crediting the sender instead of the payment address	Major	Resolved
14	Precompile ABI definitions mismatch handlers, rendering darkpool and stealth non-functional	Major	Resolved

15	<code>getSettlement</code> passes incorrect types, causing runtime panics	Major	Resolved
16	Darkpool entity ID generation diverges between EVM and SDK paths	Major	Resolved
17	<code>announce</code> ABI omits fields required by <code>MsgAnnounce</code>	Major	Resolved
18	Unshield destination fields are not cryptographically bound to the masp transaction, enabling front-running	Major	Resolved
19	Per-asset value balance collapsed into a scalar, allowing cross-asset minting	Major	Resolved
20	FFI computation cost is not gas-metered, enabling cheap denial of service of the validator wall-clock	Major	Resolved
21	Precompile error path bypasses EVM gas accounting and lacks the upstream <code>RunAtomic</code> patch	Major	Resolved
22	Darkpool order IDs collide for identical orders within the same block	Minor	Resolved
23	Missing validations in darkpool and stealth precompiles	Minor	Resolved
24	MASP precompile does not emit EVM events for state-changing operations	Minor	Resolved
25	<code>registerMetaAddress</code> accepts unvalidated stealth pubkeys	Minor	Resolved
26	RFQ user-specified <code>maxPrice</code> is discarded	Minor	Resolved
27	RFQ's EVM <code>ExpiryBlock</code> is hardcoded	Minor	Resolved
28	Settlement not-found returns zero struct instead of reverting	Minor	Resolved
29	<code>getSettlementBatch</code> accepts an unbounded input length	Minor	Resolved
30	<code>getAnnouncementsSince</code> always scans the full announcement store	Minor	Resolved
31	Genesis import drops the <code>Nullifiers</code> field, allowing every previously spent note to become re-spendable after migration	Minor	Resolved
32	Rust panics across the FFI boundary abort the	Minor	Resolved

	validator process and bypasses Cosmos SDK panic recovery		
33	Similar MASP proof verified up to three times per transaction	Minor	Resolved
34	Incorrect error handling in <code>ffi.InitParams</code>	Minor	Resolved
35	<code>profile.release</code> does not enable overflow-checks	Minor	Resolved
36	Redundant synthetic pseudo-nullifier in <code>unshieldSimplified</code> shares a KV namespace with real <code>Sapling</code> nullifiers	Minor	Resolved
37	Anchor verification accepts only the current Merkle root, with no rolling history	Minor	Resolved
38	Validator-side filesystem hardening is not enforced for MASP parameter files	Minor	Resolved
39	No denom allow-list on shield, unshield, or shielded-transfer paths allows accepting any bank denom into the MASP pool	Minor	Resolved
40	<code>try_decrypt_note</code> is an explicit stub returning <code>None</code> , preventing wallet syncs	Minor	Resolved
41	Synthetic simplified-unshield nullifier collides for same-block, same-recipient, same-amount withdrawals	Minor	Resolved
42	FFI JSON wire format carries no version byte, so silent schema drift between Rust and Go corrupts verification results	Minor	Resolved
43	Hardcoded <code>BranchId::MASP</code> in <code>Transaction::read</code> locks the chain to one <code>BranchId</code> without an upgrade path	Minor	Resolved
44	Orphan FFI exports lack any Go caller, including the safety-critical <code>hyve_masp_check_invariant</code>	Minor	Resolved
45	<code>keeper.NewKeeper</code> hardcodes <code>DefaultParams</code> , causing genesis-declared parameters to be silently overridden	Minor	Resolved
46	Caller-supplied <code>NullifierSeed</code> is silently overwritten by the keeper	Minor	Resolved
47	<code>CountOpenOrdersByUser</code> iterates the entire open-order set on every call instead of maintaining a per-user counter	Minor	Resolved

48	<code>hyve_masp_key_bundle_from_seed</code> does not validate seed entropy before deriving the spending key	Minor	Resolved
49	<code>verify_masp_transaction</code> accepts masp transactions with transparent legs, even though the chain integration ignores them	Minor	Resolved
50	<code>registerMetaAddress</code> does not implement the Label field	Informational	Resolved
51	<code>getMatchHistory</code> requires a pair argument but discards it	Informational	Resolved
52	<code>getAnnouncements</code> returns hex-encoded ASCII instead of decoded key bytes	Informational	Resolved
53	<code>shieldedBalance</code> query is pool-wide, not per-user	Informational	Resolved
54	Sapling nullifiers and shielded balances are duplicated structurally across Rust and Go state	Informational	Resolved
55	<code>ShieldedPool</code> uses <code>HashSet</code> and <code>HashMap</code> with the default <code>RandomState</code> hasher, producing non-deterministic serialization	Informational	Resolved

Detailed Findings

1. TWAP records are never pruned, allowing chain halt via submit-then-cancel spam

Severity: Critical

In `x/darkpool/module/module.go:112`, the `EndBlocker` calls `ProcessTWAPSlices` every block. `ProcessTWAPSlices` at `x/darkpool/keeper/advanced_orders.go:92` calls `IterateTWAPOrders` in lines 230–244, which iterates over every TWAP record under `TWAPOrderPrefix` and JSON-unmarshals each one before filtering by status. When a TWAP reaches a terminal state, both `CancelTWAPOrder` (lines 64–88) and the terminal transitions inside `ProcessTWAPSlices` flip only the `Status` field. No code path deletes the record from the store.

The issue is that the per-user cap `MaxActiveTWAPs = 10` only counts active TWAPs via `CountActiveTWAPsByUser` at `x/darkpool/keeper/advanced_orders.go:219–228`, so a submit-then-cancel sequence permanently adds a `Cancelled` record without counting against the cap. Both `SubmitTWAPOrder` and `CancelTWAPOrder` can be called in the same transaction without delay, and the attacker incurs no escrow cost beyond gas.

Consequently, the state will grow unbounded, and the per-block `EndBlocker` cost will increase monotonically until it exceeds the block-time budget, halting consensus, or exhausting node memory.

This issue also occurs in `x/darkpool/keeper/advanced_orders.go:479`, where `IterateIcebergOrders` follows the same pattern and is subject to the same unbounded growth.

Recommendation

We recommend deleting TWAP records on every terminal-status transition (`Cancelled`, `Completed`, `Expired`) from both `CancelTWAPOrder` and `ProcessTWAPSlices`.

Alternatively, consider moving terminal records to a separate prefix that is not walked by the `EndBlocker`. The same fix should be applied to iceberg orders at `x/darkpool/keeper/advanced_orders.go:479`.

Status: Resolved

2. Unshield validation does not bind amounts or recipients to the ZK proof, allowing full pool drainage

Severity: Critical

In `precompiles/common/masp_tx_helpers.go:123-178`, `ValidateMaspUnshieldCall` verifies the ZK proof for an unshield operation. The keeper's `Unshield` at `x/masp/keeper/keeper.go:248-324` then decrements the global `ShieldedBalance` and sends tokens to `msg.Recipient`.

The issue is that `ValidateMaspUnshieldCall` never compares `msg.Amount` to the proof's `result.ValueBalance` and/or never checks that `msg.Recipient` is bound to the proof's public inputs. This means anyone with any valid unshield proof can set `msg.Amount` to the entire pool balance and `msg.Recipient` to themselves.

Consequently, every validation gate will pass, and the chain will pay out in a single transaction. The keeper only decrements the global `ShieldedBalance` and never touches the per-user balance map, so existing auto-shielded balances will remain recorded against users whose funds have already been drained.

Recommendation

We recommend rejecting any unshield where `msg.Amount != result.ValueBalance` or where `msg.Recipient` is not committed to the proof's public inputs.

Status: Resolved

3. `SubmitOrder` accepts arbitrary orders without verifying the caller's solvency or escrowing tokens

Severity: Critical

In `x/darkpool/keeper/order.go:17-80`, `SubmitOrder` stores a new order into the darkpool order book. The precompile entry point at `precompiles/darkpool/tx.go:19-88` follows the same path. A per-account configuration cap, `MaxOpenOrdersPerUser`, limits the number of concurrent open orders.

The issue is that neither path verifies that the caller has sufficient funds or has escrowed any tokens before committing the order. The per-account cap does not deter an attacker who rotates across fresh accounts, since each new account gets a fresh allowance.

Consequently, the matching engine at `x/darkpool/keeper/matching.go:32-115` will perform $O(n)$ price-crossing on this polluted order book each `EndBlocker`. An attacker can poison advertised liquidity at no cost and force validators to perform unfunded matching work every block.

Recommendation

We recommend verifying that the caller holds the required token balance before committing the order. Escrowing the order amount at submission time would both prevent unfunded orders from entering the book and provide a natural deterrent to spam.

Status: Resolved

4. Stealth address generation uses broken cryptography

Severity: Critical

In `precompiles/stealth/stealth.go:253-271`, the `generateStealthAddressQuery` function attempts to derive a stealth address on-chain. The function hashes the block hash together with the recipient's spend and view public keys using `keccak256`, then slices the 32-byte result to 33 bytes with `h.Bytes()[:33]` and treats the truncation as an “ephemeral key”.

However, there are several issues with this implementation:

- **Runtime panic:** a panic will occur because `crypto.Keccak256Hash` returns exactly 32 bytes, so slicing to index 33 via `h.Bytes()[:33]` will cause an index-out-of-range panic, crashing the node.
- **Deterministic per block:** the output is deterministic per block because the inputs are the public block hash and publicly registered meta-address keys. Any observer can replay the derivation for every registered meta-address at every block to link recipients to stealth addresses.
- **Not a valid elliptic curve point:** the 33-byte output is a hash truncation, not a valid `secp256k1` point. Off-chain wallets using standard ECDH derivation (e.g., [EIP-5564](#)) will be unable to recover the corresponding private key.
- **No randomness:** real stealth address protocols require sender-generated ephemeral private keys as entropy, which cannot be implemented via on-chain queries.

Consequently, invoking this function will crash the node. Even if the panic were fixed, the generated addresses would still provide no privacy because any observer can deterministically reconstruct every stealth address from public on-chain data.

Recommendation

We recommend removing the on-chain `generateStealthAddress` function entirely. In a proper stealth address protocol such as EIP-5564, address derivation must be performed client-side with locally generated ephemeral secrets.

Status: Resolved

5. Shield path produces no shielded note, leaving the privacy module structurally non-functional

Severity: Critical

In `x/masp/keeper/keeper.go:199`, the `Shield` function performs a transparent transfer of `msg.Amount` from the sender to the MASP module account, increments `ShieldedBalance` and `ShieldedUserBalance`, and emits an event. The precompile entry point `shieldExec` at `precompiles/masp/masp.go:144` constructs a `MsgShield` from a payment address, denom, and amount, and passes it to `Shield`.

The issue is that `Shield` never calls `ffi.PoolAppendCommitment` and never appends any output commitment to the Sapling tree. The `MsgShield` does not carry a MASP transaction or a shielded output. In Sapling/MASP, a shield operation must insert a note tuple (value commitment, note commitment, ephemeral key, encrypted ciphertext) into the commitment tree so that the recipient receives a spendable note. Since no commitment is ever appended, the tree never grows, and no spend bundle can produce a valid Merkle authentication path.

Consequently, `Unshield` and `ShieldedTransfer` will never be able to execute against any on-chain note. The only withdrawal path is `UnshieldSimplified` at `x/masp/keeper/keeper.go:381`, which performs a transparent `bank.Send` gated by a per-user counter and offers no privacy benefit. The privacy module will be unable to reach a non-empty steady state from a clean genesis, every ZK code path will be unreachable, and the entire shielded module will remain non-functional until the shield path produces real notes.

Recommendation

We recommend reshaping `Shield` to accept a MASP transaction with one transparent input and one shielded output, verifying the bundle through the existing `ffi.VerifyMaspTx` pipeline.

The per-asset value balance should equal the negated shield amount for the given asset type. The resulting commitments should be appended to the pool tree, and the sender should be debited via the bank module.

Status: Resolved

6. Convert anchor is unbound, allowing a prover to mint shielded staking rewards and airdrop tokens at arbitrary multipliers

Severity: Critical

In `x/masp/keeper/keeper.go:444-476`, `BeginBlocker` advances the epoch every `EpochDurationBlocks` and calls `ffi.AdvanceEpoch`, which produces a list of

conversion rules in `hyve-masp/src/convert.rs:58-99`. Each rule is a tuple of input denom, output denom, rate numerator, rate denominator, and epoch. A convert spend allows a holder of the input denom to mint the output denom at the declared rate and is the sole mechanism by which legitimate per-epoch reward and airdrop multipliers reach shielded balances.

The issue is that the Go side discards the rule contents. Only the rule count is logged and emitted via `EventTypeEpochAdvance` in lines 463-474. No KV namespace stores the rule set or its Merkle root. The `verify_masp_transaction` function in `hyve-masp/src/proofs.rs:135-211` passes the convert verification key to the validator, but the convert anchor is sourced from each `ConvertDescription` in the bundle, which is supplied by the prover. The Groth16 public inputs for the convert circuit are the value commitment and anchor. Soundness binds the proof to whatever anchor the prover declares, not to any chain-side value. `VerifyResult` does not surface convert anchors back to Go, so the keeper has no data to compare against.

Consequently, a prover can construct a private conversion tree with leaves of their choosing. For example, they can declare a 100:1 entry rate instead of the legitimate 1:1 rate, or a 1000000:1 reward rate instead of the legitimate 1.0001:1 rate. They compute that tree's root, generate a Merkle path for the chosen leaf, and produce a Groth16 proof binding the value commitment to the attacker's root. The verifier checks the pairing against the convert verification key and the attacker's root, sees a sound proof, and accepts it. The chain will mint output-denom tokens at the attacker's chosen multiplier.

Applied to the staking-reward rail, a single convert spend will print arbitrarily many epoch-wrapped tokens. Applied to airdrop conversions of the same shape, it will inflate any airdrop allocation by the attacker's chosen scalar. Once unshielded, those tokens will drain the transparent reserve at face value.

Recommendation

We recommend persisting the conversion-tree anchor in KV per epoch by adding a `ConversionAnchorStoreKey(epoch)` to `x/masp/types/keys.go`.

In `BeginBlocker`, after `AdvanceEpoch` returns rules, the conversion-tree root should be computed over the rule set and written as an (epoch, root) pair to KV.

In `verify_masp_transaction`, every converted spend should be checked against the stored anchor for its declared epoch and rejected if the anchor does not match.

Status: Resolved

7. DarkPool1 precompile does not populate `order.Commitment`, causing all of a user's orders to share one nullifier

Severity: Major

In `precompiles/darkpool/tx.go:64-75`, the `submitOrder` handler constructs an order and passes it to the keeper. The keeper at `x/darkpool/keeper/order.go:38-40` derives a nullifier seed from the order's commitment and sender address. The commitment scheme is designed to provide privacy and prevent replay attacks by ensuring each order has a unique nullifier.

The issue is that the precompile never sets the `Commitment` field on the order. When the commitment is empty, the keeper derives the nullifier seed as the hash of the sender address alone, which collapses to a single value per sender across all orders.

Additionally, the keeper never calls `RevealNullifier`, so the nullifier mechanism does not guard against replay attacks or double-spending. Any order's nullifier will match every other order that the same sender creates.

Consequently, the privacy-preserving guarantee of the commitment scheme is removed, and the intended settlement deduplication will fail. All orders from the same sender will share a single nullifier, allowing replay attacks and breaking the design assumption that each order is uniquely identifiable.

Recommendation

We recommend requiring the precompile to supply a caller-signed commitment for each order, return an error if it is empty, and call `RevealNullifier` at settlement time.

Status: Resolved

8. ExpireRFQs scans every RFQ every block

Severity: Major

In `x/darkpool/keeper/rfq.go:110`, `ExpireRFQs` is called from the `EndBlocker` to expire stale RFQs. The function iterates over all RFQs in the store and checks their expiry status.

The issue is that there is no expiry index or pruning mechanism. The function walks every RFQ that has ever been created, so the per-block cost is proportional to the total number of RFQs in the state. An attacker can submit cheap RFQs to permanently inflate the per-block work.

Consequently, the `EndBlocker` cost will grow unbounded as RFQs accumulate. After enough RFQ spam, the iteration will exceed the block-time budget, halting consensus.

Recommendation

We recommend maintaining a secondary index keyed by expiry height and deleting RFQs once they expire.

Status: Resolved

9. DarkPool keeper events leak plaintext order, RFQ, and match details

Severity: Major

In `x/darkpool/keeper/order.go:67-77`, `SubmitOrder` emits events containing the pair, side, amount, price, and sender. The same pattern occurs for RFQ and match events throughout the keeper.

Since events are part of the public block data that every full node and explorer indexes, emitting plaintext order details destroys the darkpool privacy property for every order submitted via this path. An observer can reconstruct the full order book in real time without running any off-chain infrastructure.

Consequently, the darkpool will offer no privacy protection against any participant who monitors the chain. The design assumption that order book privacy is maintained through commitment-based nullifiers will be invalidated.

Recommendation

We recommend removing plaintext order fields from events and emitting only commitment hashes and opaque IDs. Additionally, consider surfacing richer data exclusively via authenticated RPCs.

Status: Resolved

10. SubmitIcebergOrder and SubmitTWAPOrder emit the hidden total amount, defeating the privacy feature

Severity: Major

In `x/darkpool/keeper/advanced_orders.go:305-314`, `SubmitIcebergOrder` emits an `iceberg_submitted` event. The defining property of an iceberg order is that observers should only see the `VisibleAmount`, while the total order size remains hidden.

The issue is that the event's amount attribute is set to `iceberg.TotalAmount`, the very value the iceberg is supposed to hide. Since events are part of the public block data, any indexer or block scanner can reconstruct the full size for free.

This issue also occurs in `x/darkpool/keeper/advanced_orders.go:49-58`, where `SubmitTWAPOrder` emits the full `TotalAmount`, exposing the complete schedule size on submission.

Consequently, the privacy guarantees of iceberg and TWAP orders will be defeated. Observers will be able to determine the full order size despite the design intent to keep it hidden.

Recommendation

We recommend emitting only the order ID, pair, side, sender, and `VisibleAmount` for iceberg orders. Additionally, consider exposing `TotalAmount` and `SliceCount` only through authenticated, owner-scoped queries.

Status: Resolved

11. Settlement verifier output is always zero because `HexToAddress` is applied to bech32

Severity: Major

In `precompiles/settlement/query.go:47-50`, `getSettlement` returns the verifier address by calling `common.HexToAddress(receipt.Verifier)`. The verifier is set at two write sites: `InitializeSettlement` at `x/settlement/keeper/settlement.go:30` uses `k.GetAuthority`, and `MsgUpsertSettlement` at `x/settlement/keeper/msg_server.go:36` uses the message authority. Both write sites populate the field with a Cosmos bech32 string.

The issue is that `HexToAddress` expects hex-encoded input, but the verifier is always a bech32 string. The `Hex2Bytes` function silently swallows the decode error and returns zero bytes, so `HexToAddress` always returns the zero address.

Consequently, any downstream authorization or audit workflow that relies on the verifier field will fail. Consumers cannot identify the verifier responsible for a settlement, as the field will always be the zero address regardless of which authority is actually responsible.

Recommendation

We recommend decoding `Verifier` via `sdk.AccAddressFromBech32` and converting to `common.Address` via `common.BytesToAddress(addr.Bytes())`, returning an explicit error when the bech32 decode fails.

Alternatively, consider storing `Verifier` as raw bytes in the receipt and skip the conversion.

Status: Resolved

12. `getSettlementBatch` returns JSON bytes that Solidity cannot decode

Severity: Major

In `precompiles/settlement/query.go:96-120`, `getSettlementBatch` marshals the receipt slice with `json.Marshal` in line 114 and returns the resulting byte blob packed into a bytes ABI slot in line 119. The ABI at `precompiles/settlement/abi.json:42` declares the return type as bytes rather than a structured tuple array.

Since Solidity has no native JSON decoder, the response becomes opaque to on-chain consumers. Additionally, Go's `json.Marshal` encodes byte fields such as `MatchID`, `ReceiptHash`, and `ProofHash` as base64 strings, so even a third-party JSON parser would require an additional base64 decode to recover the raw bytes.

Consequently, the method will be unusable for contract callers and will force every integration to use off-chain decoding.

Recommendation

We recommend declaring the return type as a structured tuple array matching the receipt shape and encoding with `abi.Pack`.

Status: Resolved

13. MASP Shield ignores `ShieldedRecipient`, crediting the sender instead of the payment address

Severity: Major

In `x/masp/keeper/keeper.go:199-245`, the `Shield` function credits the per-user shielded balance to `msg.Sender` in lines 226 and 232. The precompile's `BuildMaspShieldMsg` in `precompiles/common/masp_tx_helpers.go:43` plumbs the EVM payment address into the `ShieldedRecipient` field, and `ValidateBasic` at `x/masp/types/msgs.go:19` requires it to be non-empty. This is intended to support custodians, relays, or deposit-for flows where a user shields on behalf of another address.

The issue is that the keeper never references `msg.ShieldedRecipient`. The field reaches the keeper, but is eventually discarded. When a user attempts to shield on behalf of another address (e.g., a common pattern for custodians, relays, or “deposit-for” flows), the shielded credit is applied to the sender instead.

Consequently, a sender can withdraw the tokens via the simplified-unshield path before the intended recipient ever sees them. This is a fund misrouting rather than a no-op argument.

Recommendation

We recommend reading `msg.ShieldedRecipient` in the keeper and using it as the key for `SetUserShieldedBalance`, with a fallback to `msg.Sender` only if the field is empty.

Additionally, consider updating `ValidateBasic` to reject mismatched representations between the bech32 sender and recipient address forms.

Status: Resolved

14. Precompile ABI definitions mismatch handlers, rendering darkpool and stealth non-functional

Severity: Major

In `precompiles/darkpool/abi.json`, 9 of the function signatures define argument counts and types that are incompatible with the corresponding handlers in `precompiles/darkpool/tx.go`, `precompiles/darkpool/query.go`, `precompiles/darkpool/tx_phase3.go`, and `precompiles/darkpool/query_phase3.go`.

Specifically, the go-ethereum ABI decoder unpacks calldata according to the JSON ABI, producing a typed argument slice that the precompile then casts to its expected types. When the ABI and the precompile disagree on count or types, the handler either receives the wrong number of arguments and rejects them, or the type assertion fails and returns an error.

For example, `submitOrder` in the `precompiles/darkpool/abi.json:11-16` decodes 6 inputs (`bytes32`, `string`, `string`, `uint256`, `uint256`, `string`), but `precompiles/darkpool/tx.go:19-27` expects 5 inputs of types (`string`, `bool`, `*big.Int`, `*big.Int`, `[32]byte`). The remaining functions (`cancelOrder`, `createRFQ`, `acceptRFQ`, `cancelRFQ`, `openDispute`, `getOrder`, `getMatch`, `getMatchHistory`) each have analogous count or type mismatches.

On the other hand, in the `stealth` precompile, `generateStealthAddress` in `precompiles/stealth/abi.json:98-104` declares 1 input of type `address`, but the handler at `precompiles/stealth/stealth.go:240-242` expects 2 inputs of type `[]byte`. The decoder produces one `common.Address` argument, so the `len(args) != 2` guard will always fail.

Consequently, the entire `darkpool` precompile, and the `stealth` precompile's `generateStealthAddress` function will be non-functional at runtime. Any Solidity contract or off-chain tool encoding calls against these ABIs will always receive errors.

Recommendation

We recommend aligning the ABI definitions with the handler signatures for every function.

Status: Resolved

15. `getSettlement` passes incorrect types, causing runtime panics

Severity: Major

In `precompiles/settlement/query.go:34`, the `getSettlement` function packs a not-found response by passing an empty string (`""`) for the `settlementTxHash` parameter output.

However, the ABI declares the `settlementTxHash` parameter output as `bytes32` in `precompiles/settlement/abi.json:15`. This means that `go-ethereum abi.Pack` method requires a `[32]byte` and will reject the given string. This issue also exists in `precompiles/settlement/query.go:56`, where the handler passes `receipt.SettlementTxHash` as a `[]byte` instead of `[32]byte`.

Additionally, the `matchedBlock`, `settlementBlock`, and `updatedAt` response parameters are passed as `*big.Int`, but the ABI declares them as `uint64` in `precompiles/settlement/abi.json:17-19`.

Consequently, these type mismatches cause `abi.Pack` to return an error on every invocation. Any Solidity contract or off-chain tool that calls `getSettlement` will always receive an error, rendering settlement receipt queries unusable on the EVM.

Recommendation

We recommend implementing the following recommendations:

1. Convert `receipt.SettlementTxHash` to a `[32]byte` via `copy()` before packing
2. Use a zero-value `[32]byte{}` in the not-found path (`!found`)
3. Pass block numbers and timestamps as `uint64` values directly, rather than wrapping them in `big.NewInt`.

Status: Resolved

16. Darkpool entity ID generation diverges between EVM and SDK paths

Severity: Major

The EVM precompile in `precompiles/darkpool/*.go` and the SDK in `x/darkpool/keeper/msg_server.go` use different algorithms to generate IDs for the same entity types:

- **Order ID:** the precompile at `precompiles/darkpool/tx.go:56-62` hashes (`caller_bytes || blockHeight || pair || amount || price`) as raw binary. The SDK at `x/darkpool/keeper/msg_server.go:24-25` uses a client-supplied `msg.OrderID` without enforcing a similar generation mechanism.
- **RFQ ID:** the precompile at `precompiles/darkpool/tx.go:162-165` hashes (`caller_bytes || pair || amount || blockHeight`) as a raw binary string. The SDK at `x/darkpool/keeper/msg_server.go:50-52` hashes

"rfq|creator_bech32|pair|height|time" and hex-encodes the first 16 bytes.

- **TWAP ID:** the precompile at `precompiles/darkpool/tx_phase3.go:71-72` hashes (`caller_bytes || "twap|pair|height"`) and hex-encodes the first 16 bytes. The SDK at `x/darkpool/keeper/msg_server.go:104-106` includes `BlockTime` in the hash input, which the EVM path omits.
- **Iceberg ID:** the precompile at `precompiles/darkpool/tx_phase3.go:172-173` hashes (`caller_bytes || "iceberg|pair|height"`) and hex-encodes the first 16 bytes. The SDK at `x/darkpool/keeper/msg_server.go:136-138` includes `BlockTime`, which the EVM path omits.

Consequently, these divergences cause the EVM and SDK paths to produce different IDs for the same logical entity, breaking cross-path interoperability for all darkpool entity types. For example, an order created via the EVM precompile cannot be canceled via the SDK transaction path, and vice versa.

Recommendation

We recommend unifying ID generation in the keeper layer. The keeper's `SubmitOrder`, `CreateRFQ`, `SubmitTWAPOrder`, and `SubmitIcebergOrder` should generate deterministic IDs internally using a single consistent algorithm.

Status: Resolved

17. announce ABI omits fields required by MsgAnnounce

Severity: Major

In `precompiles/stealth/stealth.go:201-206`, the `announceExec` handler constructs a `MsgAnnounce` with only `Sender`, `EphemeralPubKey`, `StealthAddress`, and `ViewTag`.

However, the SDK's `MsgAnnounce` in `x/stealth/types/msg.go:35-54` supports two more parameters: `Amount` and `Denom`. The `ValidateBasic` validation ensures that the `Amount` must be positive and `Denom` must be non-empty in `x/stealth/types/msg.go:48-53`.

Since the precompile does not include the `amount` or `denom` parameters, `MsgAnnounce` will be constructed with a zero-value `Amount` and an empty `Denom`. This will cause the transaction to fail in `x/stealth/keeper/msg_server.go:34`.

With that said, the keeper's `Announce` function in `x/stealth/keeper/keeper.go:97-129` does not actually read the `Amount` and `Denom` fields, suggesting the SDK message definition has drifted from the implementation.

Recommendation

We recommend implementing either of the recommendations:

- If the `Amount` and `Denom` fields are required, implement them in the ABI, precompile handler, and the relevant use case in the keeper's `Announce` function.
- If the `Amount` and `Denom` fields are not required, remove them from the `MsgAnnounce` struct and its accompanying `ValidateBasic` validation.

Status: Resolved

18. Unshield destination fields are not cryptographically bound to the masp transaction, enabling front-running

Severity: Major

In `precompiles/common/masp_tx_helpers.go:104`, `BuildMaspUnshieldMsg` and `ValidateMaspUnshieldCall` construct an `MsgUnshield` message from a MASP transaction, recipient, denom, and amount. The `Unshield` function at `x/masp/keeper/keeper.go:248` then sends the specified amount of tokens to the recipient via the bank module. The sighash construction in `hyve-masp/src/proofs.rs:148` uses the transaction ID shortcut with no transparent inputs, so the spend authorization signatures and the binding signature cover only the MASP transaction contents, not the wrapping Cosmos message.

The issue is that the recipient, denom, and amount are not cryptographically bound to the MASP transaction. The keeper derives the destination fields from the `MsgUnshield` message instead of from the MASP transaction itself. A mempool observer can copy a victim's MASP transaction bytes, wrap them in a new `MsgUnshield` with their own recipient, and submit it with higher gas. The chain processes the attacker's transaction first, but the MASP signatures still verify against the unchanged bundle, causing the bank module to send funds to the attacker.

Consequently, the victim's later transaction will fail the duplicate-nullifier check, and the victim will lose the spent notes and the gas paid. The front-running vulnerability allows any observer to steal unshielded funds by simply wrapping the victim's MASP transaction with a different recipient.

Recommendation

We recommend implementing the following suggestions:

- Moving the destination fields into the MASP transaction as a transparent output encoding of recipient, asset type, and amount
- Modifying `VerifyResult` in `x/masp/ffi/masp_bridge.go:21` to surface the parsed transparent output to the Go side.
- Update `keeper.Unshield` to derive the recipient, denom, and amount from the MASP transaction rather than from `MsgUnshield`.

Status: Resolved

19. Per-asset value balance collapsed into a scalar, allowing cross-asset minting

Severity: Major

In `hyve-masp/src/proofs.rs:188-191`, the `verify_masp_transaction` function reduces the per-asset value balance map into a single `i128` scalar by summing all amounts and discarding the asset key. The result is narrowed and serialized as `VerifyResult.ValueBalance (int64)` in `x/masp/ffi/masp_bridge.go:25`. Every Go-side consumer, including `ValidateMaspShieldedTransferCall` at `precompiles/common/masp_tx_helpers.go:201`, the ante decorator at `x/masp/ante/ante.go:211`, and `ShieldedTransfer` at `x/masp/keeper/keeper.go:326` sees only this scalar.

The issue is that the MASP binding signature attests that the per-asset sum of value commitments equals `valueBalance[t] · G_t` for each asset type. The consensus rule the chain must enforce is that the per-asset map is the zero vector, but the implementation enforces only that the scalar sum is zero.

An attacker who holds shielded notes of two different assets can construct a `shieldedTransfer` whose declared `value_balance` is a positive amount for one asset and a negative amount for the other (e.g., `{TokenA: +N, TokenB: -N}`), the binding signature verifies, the scalar sum is zero, and the chain accepts the transfer as balanced.

Consequently, the attacker can convert asset A to asset B and drain the pool of any high-value asset down to the cost of their cheapest holdings.

Recommendation

We recommend replacing the scalar accumulator with a per-asset map (`BTreeMap<[u8; 32], i64>` keyed by `AssetType.get_identifier`) and updating `VerifyResult` to carry the typed map.

In `ValidateMaspShieldedTransferCall`, we recommend requiring that every per-asset entry be zero.

In `ValidateMaspUnshieldCall` and `ValidateMaspShieldCall`, we recommend requiring exactly one entry whose 32-byte identifier matches the canonical `AssetType` for the message denom.

Status: Resolved

20. FFI computation cost is not gas-metered, enabling cheap denial of service of the validator wall-clock

Severity: Major

In `precompiles/common/precompile.go:38-44`, the `RequiredGas` function returns only `KvGasConfig.WriteCostFlat + WriteCostPerByte * len(input)` for transactions and provides no term that reflects the cost of the underlying ZK work. The `Precompile.RequiredGas` override in `precompiles/masp/masp.go:73-85` defers entirely to the base implementation. The `VerifyMaspTx` wrapper at `x/masp/ffi/masp_bridge.go:217-239` calls into Rust over `cgo` without bracketing the call with gas consumption tracking.

The issue is that the Cosmos SDK gas meter is a Go-side accounting object and is invisible to anything running inside an extern “C” Rust function. The MASP module's expensive work, including bundle deserialization, batch validation checks (e.g., `BatchValidator::check_bundle`), BLS12-381 multi-pairing operations proportional to `n_spend + n_outputs + n_converts`, and Merkle tree appends, all happen on the Rust side.

Consequently, an attacker can submit transactions whose `MaspTx` field passes the cheap front-half checks but fails `BatchValidator::validate`, costing only the precompile flat gas while burning 10-50 ms of validator wall-clock time per verification. On a chain with 30M gas per block and a 1 s block time, the attacker can make approximately 240 attempts per block and incur 7-12 seconds of pure verification load per block.

Recommendation

We recommend overriding `RequiredGas` in `precompiles/masp/masp.go` to add a verification-cost term parameterized by the bundle counts, charging `BaseVerifyGas + PerSpendGas * n_spend + PerOutputGas * n_outputs + PerConvertGas * n_converts`.

Additionally, consider adding explicit `ctx.GasMeter().ConsumeGas` calls inside the keeper, ante decorator, and validator paths so that `MsgUnshield` envelopes submitted directly to the message server are also charged.

Lastly, consider enforcing per-component limits before the gas charge to bound the worst-case per-transaction work.

Status: Resolved

21. Precompile error path bypasses EVM gas accounting and lacks the upstream `RunAtomic` patch

Severity: Major

In `precompiles/common/precompile.go:108-117`, the `runNativeAction` function returns in line 110 immediately when the action returns a non-nil error. This happens before reaching the gas consumption computation (`cost := ctx.GasMeter().GasConsumed() - initialGas`) in line 113 and `contract.UseGas` call in line 115.

The deferred `HandleGasError` function in `precompiles/common/precompile.go:181` triggers only on a recoverable out-of-gas panic (`storetypes.ErrorOutOfGas`) and does not fire on a normal return error.

The issue is that users pay only the required gas for any failed precompile invocation, regardless of how much SDK-side work was performed.

Consequently, failed precompile calls will not account for the actual work performed, allowing gas underpayment.

Recommendation

We recommend charging the consumed gas on the early return paths.

Status: Resolved

22. Darkpool order IDs collide for identical orders within the same block

Severity: Minor

In `precompiles/darkpool/tx.go:56-62`, the order hash is computed from `(caller, blockHeight, pair, amount, price)`. The `nullifierHash` parameter, which is unique per order, is not included in the hash input.

If the same user submits two orders with identical pair, amount, and price in the same block, both will produce the same `orderHash`. The keeper's `Set` performs an upsert, so the second order will silently overwrite the first in the KV store.

This issue also occurs in the following instances:

- `precompiles/darkpool/tx_phase3.go:71-72`: TWAP IDs are derived from `sha256(caller || "twap|pair|height")`, so two TWAP orders for the same pair in the same block from the same user collide.
- `precompiles/darkpool/tx_phase3.go:172-173`: Iceberg IDs use the same pattern with `"iceberg|pair|height"`, producing identical collisions.

Consequently, users will silently lose orders. A user who submits two buy orders for the same pair and amount in a single block will receive only the second order, with no error or indication that the first was canceled.

Recommendation

We recommend including the `nullifierHash` or commitment in the order hash input to guarantee uniqueness, and adding a monotonic counter or transaction index for TWAP and iceberg IDs.

Additionally, the keeper should check for an existing record before writing and return an error on collision rather than performing a blind upsert.

Status: Resolved

23. Missing validations in darkpool and stealth precompiles

Severity: Minor

In `precompiles/darkpool/tx.go:19-267` and `precompiles/darkpool/tx_phase3.go:18-254`, all write handlers construct raw domain types such as `Order{}`, `RFQRecord{}`, `TWAPOrder{}`, and `IcebergOrder{}` and pass them directly to keeper methods without calling `ValidateBasic`. The `registerMetaAddressExec` and `announceExec` functions in `precompiles/stealth/stealth.go:126-211` also do not call `ValidateBasic`.

This is problematic because the SDK-level constraints defined in `x/darkpool/types/messages.go:22-415` will not be fully enforced in EVM. Some examples are listed below:

- **submitOrder**: the EVM checks `amount > 0` and `price > 0`, but does not check the `ValidPairs` allowlist (see `x/darkpool/types/messages.go:26`) or validate `Side`.
- **createRFQ**: the EVM checks `amount > 0` and `minPrice >= 0`, but does not check `ValidPairs`, `Side`, and `Commitment` length.
- **submitTWAPOrder**: the EVM checks `totalAmount > 0`, `sliceCount > 0`, `intervalBlocks > 0`, and `Side` validity, but does not enforce `TWAPMinSlices/TWAPMaxSlices` or `TWAPMinInterval/TWAPMaxInterval` bounds (see `x/darkpool/types/messages.go:259-263`).
- **submitIcebergOrder**: the EVM checks `totalAmount > 0`, `visibleAmount > 0`, `visibleAmount <= totalAmount`, and `Side` validity, but does not enforce the 5–50% visibility percentage range (see `x/darkpool/types/messages.go:330-334`).
- **announce**: the SDK checks `ViewTag > 255` in `x/stealth/types/msg.go:45`, but the EVM path accepts any `uint256` value and truncates via `uint32(viewTag.Uint64())`.

Consequently, EVM callers are able to submit orders with invalid pairs, TWAP orders with unbounded slice counts or intervals, iceberg orders violating visibility percentage constraints, and stealth announcements with out-of-range view tags.

Recommendation

We recommend constructing `Msg*` types and calling `ValidateBasic` in every write handler before passing them to the keeper, similar to the MASP precompile implementation in `precompiles/common/masp_tx_helpers.go`.

Status: Resolved

24. MASP precompile does not emit EVM events for state-changing operations

Severity: Minor

In `precompiles/masp/masp.go:144-180`, `precompiles/masp/masp.go:183-255`, and `precompiles/masp/masp.go:257-274`, the `shield`, `unshield`, and `shieldedTransfer` handlers perform state-changing operations but do not call `stateDB.AddLog` to emit EVM events.

While the keeper emits Cosmos SDK events, these events are invisible to EVM-level indexers.

Recommendation

We recommend emitting EVM events via `stateDB.AddLog` for state-changing MASP operations.

Status: Resolved

25. `registerMetaAddress` accepts unvalidated stealth pubkeys

Severity: Minor

In `x/stealth/keeper/keeper.go:38-79`, `RegisterMetaAddress` only checks that `SpendingPubKey` and `ViewingPubKey` are non-empty strings.

The issue is that there are no validations enforcing a `secp256k1` length, not decompressing to verify curve membership, and not rejecting `SpendingPubKey == ViewingPubKey` (which collapses the privacy property because the view key would also be the spend key).

Recommendation

We recommend rejecting any pubkey that is not 33 or 65 bytes, decompressing to confirm it lies on the `secp256k1` curve, normalizing the on-disk encoding (preferably raw bytes), and returning an error when `SpendingPubKey == ViewingPubKey`.

Status: Resolved

26. RFQ user-specified maxPrice is discarded

Severity: Minor

In `precompiles/darkpool/tx.go:149-154`, the `createRFQ` handler validates that `maxPrice` is non-negative and greater than or equal to `minPrice`.

However, the `maxPrice` value is then never stored after validation. It is discarded and not used when constructing the `RFQRecord` struct.

Consequently, users who rely on `maxPrice` to express a price range for their RFQ will find that only the lower bound is stored. Responders will have no visibility into the caller's maximum acceptable price.

Recommendation

We recommend storing `maxPrice` as a separate field in the `RFQRecord` and validating it against `minPrice`. The SDK's `MsgCreateRFQ` should also support implementing a `msg.maxPrice` to be consistent with the EVM.

Status: Resolved

27. RFQ's EVM ExpiryBlock is hardcoded

Severity: Minor

In `precompiles/darkpool/tx.go:175`, the `createRFQ` handler hardcodes `ExpiryBlock` as 100 in the `RFQRecord`. The SDK path at `x/darkpool/keeper/msg_server.go:62` uses `msg.ExpiryBlocks`, allowing users to specify their own expiry window.

Additionally, `CreateRFQ` in `x/darkpool/keeper/rfq.go:30-34` caps `rfq.ExpiryBlock` to 1000000 and convert it from relative to absolute, but the precompile already hardcodes `ExpiryBlock = 100`, so the cap is unreachable via the EVM path.

Consequently, EVM users are unable to set RFQ windows to be shorter or longer. All RFQs created via the precompile expire after exactly 100 blocks, regardless of the caller's intent. This creates an inconsistency with the SDK path, where expiry is user-configurable.

Recommendation

We recommend accepting an `expiryBlocks` parameter from the caller and validating it against a reasonable range before storing it.

Status: Resolved

28. Settlement not-found returns zero struct instead of reverting

Severity: Minor

In `precompiles/settlement/query.go:28-41`, `getSettlement` returns an all-zero tuple when the receipt does not exist instead of reverting or returning an explicit `found` flag.

This is problematic because callers cannot distinguish “no such settlement” from “settlement exists with zero values”, which is a legitimate edge case for freshly-created receipts.

Recommendation

We recommend reverting with a descriptive error (or adding an explicit `exists bool` field to the ABI) when the receipt is not found.

Status: Resolved

29. `getSettlementBatch` accepts an unbounded input length

Severity: Minor

In `precompiles/settlement/query.go:96-120`, `getSettlementBatch` does not `cap len(matchIDs)` before allocating per-entry buffers and looking up receipts. This means a caller passing a very large array can degrade query response time and node performance.

Recommendation

We recommend capping `len(matchIDs)` (e.g., 128) and returning an error when exceeded.

Status: Resolved

30. `getAnnouncementsSince` always scans the full announcement store

Severity: Minor

In `x/stealth/keeper/keeper.go:132-148`, `GetAnnouncementsSince` iterates the entire `AnnouncementStoreKey` prefix and filters by `fromHeight` in memory, so the iteration cost is $O(n)$ regardless of input.

Additionally, the precompile at `precompiles/stealth/stealth.go:214-224` also forwards the caller's `*big.Int` via `Int64()` with no range check.

Consequently, the cost increases monotonically as announcements accumulate, and a single call with `sinceBlock = 0` returns the entire history.

Recommendation

We recommend changing the keeper to seek by `fromHeight` (or maintaining a height-indexed sub-prefix) and enforcing a maximum lookback window at the precompile.

Status: Resolved

31. Genesis import drops the `Nullifiers` field, allowing every previously spent note to become re-spendable after migration

Severity: Minor

In `x/masp/types/genesis.go:6-11`, the `GenesisState` struct declared includes a `Nullifiers [][]byte` field alongside `Params`, `PoolState`, and `CurrentEpoch`. The `InitGenesis` function in `x/masp/module/module.go:103-117` reads only `gs.PoolState` and `gs.CurrentEpoch`. The `ExportGenesis` function in `x/masp/module/module.go:119-130` constructs the export using only `Params`, `PoolState`, and `CurrentEpoch`.

The issue is that the `gs.Nullifiers` slice is never iterated during initialization, so no entries are written to the `NullifierStoreKey` namespace at chain start. During export, the code never iterates the `NullifierPrefix` KV namespace, so the exported `Nullifiers` slice is always empty. After any chain upgrade or genesis-from-snapshot, the entire revealed-nullifier set is silently dropped.

Consequently, every previously spent shielded note will become re-spendable. An attacker who held shielded notes at the snapshot height can replay every spend transaction they previously executed, draining the pool of the cumulative value of all notes ever spent.

Recommendation

We recommend implementing the following suggestions:

- Iterate `gs.Nullifiers` in `InitGenesis` and calling `keeper.RevealNullifier` for each entry
- Iterate the `NullifierPrefix` KV namespace in `ExportGenesis` and append each nullifier to the exported slice
- Add a regression test that round-trips a populated nullifier set through `Export` and then `Init`.

Status: Resolved

32. Rust panics across the FFI boundary abort the validator process and bypasses Cosmos SDK panic recovery

Severity: Minor

The `#[no_mangle] pub unsafe extern "C" fn` exports in `hyve-masp/src/ffi.rs` are not wrapped in `std::panic::catch_unwind`. Rust's default toolchain treats panics propagating across an extern "C" boundary as undefined behavior: the runtime calls `std::process::abort` rather than allowing the unwind to reach C or Go stack frames. Eventually, the result is a SIGABRT to the entire hyved process rather than a Go panic or an EVM revert.

This breaks the standard Cosmos SDK panic-recovery pattern. Specifically, `BaseApp.runTx` wraps every transaction handler in a deferred recover call, but recover is a Go runtime mechanism and cannot intercept SIGABRT. A single bad transaction or a single corrupted parameter file crashes every validator that processes it. If the offending input persists in the mempool or on disk, validators that restart encounter it again in the next block, causing a persistent halt until the input is removed or the binary is patched.

This finding is a prerequisite for other recommendations in this report (notably the `overflow-checks` finding and the `InitParams` error-handling finding), which are only safe to apply once cross-FFI panics are converted to typed error codes.

Recommendation

We recommend wrapping the body of every extern "C" export in `std::panic::catch_unwind`, converting any panic into a clean error code.

Additionally, we recommend auditing all `unwrap`, `expect`, slice indexing, and arithmetic operators in `hyve-masp/src/` for potential panics when given attacker-influenced inputs, and replacing them with fallible variants that return typed errors.

Status: Resolved

33. Similar MASP proof verified up to three times per transaction

Severity: Minor

A single `MsgUnshield` or `MsgShieldedTransfer` traverses three independent verification points before the keeper completes state mutation:

- The ante decorator calls `ffi.VerifyMaspTx` in `x/masp/ante/ante.go:146` and in line 192.
- The precompile validator calls `ffi.VerifyMaspTx` in `precompiles/common/masp_tx_helpers.go:160` and 194.
- The keeper calls `ffi.VerifyMaspTx` again in `x/masp/keeper/keeper.go:266` and in line 336.

Each call performs a full Groth16 batch verification plus RedJubjub signature checks on identical (`maspTx`, `anchor`) inputs and returns identical `VerifyResult` values. The verifier is the dominant cost in the `BatchValidator::validate` module, which performs a multi-pairing of size proportional to `4 + n_spends + n_outputs + n_converts` and

consumes 10-50 ms of wall-clock time for a typical bundle. Tripling that hits 30-150 ms per transaction.

On a chain doing 100 MASP transactions per block, the difference is approximately 5 s versus 1.5 s of pure verification overhead.

Recommendation

We recommend calling the `VerifyMaspTx` once and avoiding calling it in `AnteHandler`.

Status: Resolved

34. Incorrect error handling in `ffi.InitParams`

Severity: Minor

In `x/masp/keeper/keeper.go:38-49`, the `NewKeeper` function calls `ffi.InitParams` to load the MASP ZK parameter files. If the call returns a non-OK error code (e.g., because a parameter file is missing), the keeper logs an error line and returns the constructed `Keeper` regardless, and the chain continues to start.

However, subsequent `VerifyMaspTx` calls then fail because the verifying-key cache `VERIFYING_KEYS` in `hyve-masp/src/proofs.rs:38-42` was never populated. Many failure modes in the upstream `masp_proofs::load_parameters` panic via `expect` rather than returning an error (e.g., size mismatch, hash mismatch, malformed Groth16, file unreadable), and panics across the extern "C" boundary abort the `hyved` process. The path described here is the graceful error path where Rust returns an error code, and the chain proceeds with a non-functional verifier.

Recommendation

We recommend panicking at `keeper.NewKeeper` if `ffi.InitParams` returns an error.

Alternatively, consider setting a flag on the keeper that disables MASP-related precompile methods and rejects them with an explicit “*MASP not initialized*” error rather than letting them fail later at the verify step.

Status: Resolved

35. `profile.release` does not enable overflow-checks

Severity: Minor

The `[profile.release]` section in `hyve-masp/Cargo.toml:31-34` does not set `overflow-checks`. The default value is `false` for release builds, which means that integer overflow wraps silently rather than panicking. For a chain that handles user-influenced value

sums, silent wrapping is a worse failure mode than a noisy panic, as incorrect values flow through subsequent checks undetected.

For example, the value-balance accumulator in `hyve-masp/src/proofs.rs:186` computes `value_balance += *amount`, which performs an `i128 += i128` operation and wraps on overflow.

This is unreachable as an exploit today because the `Sapling` per-note value is capped at `u64`, so reaching `i128::MAX` requires impossible bundle sizes. The recommendation to update the profile is a defense-in-depth against future code that introduces new accumulation paths.

Recommendation

We recommend adding `overflow-checks = true` to the `[profile.release]` section.

Status: Resolved

36. Redundant synthetic pseudo-nullifier in `unshieldSimplified` shares a KV namespace with real `Sapling` nullifiers

Severity: Minor

The `unshieldSimplified` path in `x/masp/keeper/keeper.go:408-413` writes a synthetic pseudo-nullifier through the same `RevealNullifier` API used for real `Sapling` nullifiers in lines 118-125. Both kinds are stored under the single `NullifierStoreKey` prefix declared in `x/masp/types/keys.go`.

The synthetic write is redundant. Double-spend in this path is already prevented by the balance accounting in lines 391-395 and 416, where the keeper rejects the call if `userShieldedBalance < amount` and decrements the balance on success. The nullifier write adds no protection that the balance check does not already provide. Storing it under the same prefix as real `Sapling` nullifiers conflates two semantically different constructs (cryptographic PRF outputs versus deterministic hashes of public data) and creates code-hygiene confusion for future readers.

Recommendation

We recommend introducing separate KV prefixes `NullifierSaplingStoreKey` and `NullifierSimplifiedStoreKey`, splitting `RevealNullifier` into `RevealSaplingNullifier` and `RevealSimplifiedNullifier`, and modifying `isNullifierSpent` to expose the kind.

Additionally, consider removing the simplified path entirely.

Status: Resolved

37. Anchor verification accepts only the current Merkle root, with no rolling history

Severity: Minor

In `hyve-masp/src/proofs.rs:170-178`, the `verify_masp_transaction` function rejects any spend whose anchor is not byte-equal to the on-chain root passed in. The `GetMerkleRoot` function in `x/masp/keeper/keeper.go:97-105` returns `ffi.PoolRoot(poolState)` for the current pool state and provides no historical lookup.

However, the codebase contains no `historicalRoots` KV namespace, as only the latest `PoolStateKey` is stored. In Sapling/Zcash and MASP, the spend circuit proves that a spent note's cmu is a member of the commitment Merkle tree at some historical anchor, not necessarily the current tip. Real Sapling keeps a rolling window of the last 100 roots in consensus state. If a shielded transfer lands at height H and changes the root, any concurrent proof built against the height H-1 root is invalid at height H.

An attacker submitting one cheap shielded operation per block prevents all other users' shielded operations indefinitely; legitimate users see their transactions fail with “anchor mismatch” through no fault of their own.

Recommendation

We recommend adding a rolling-roots KV namespace, `HistoricalRootStoreKey(blockHeight) -> root`, that stores the current root at the end of every block and retains the last K heights (suggested initial value: 100).

Additionally, consider introducing `IsValidHistoricalRoot(ctx, root []byte) bool` and modifying the anchor check in `hyve-masp/src/proofs.rs:170` to defer to Go, or updating the Go-side `VerifyMaspTx` wrapper to accept a slice of valid anchors.

Status: Resolved

38. Validator-side filesystem hardening is not enforced for MASP parameter files

Severity: Minor

The default MASP ZK parameter paths, declared in `x/masp/types/params.go:34-36`, are loaded by the keeper at chain startup. `NewKeeper` in `x/masp/keeper/keeper.go:38` and opened by the upstream `masp_proofs::load_parameters` reached via `hyve-masp/src/proofs.rs:55-95`.

Content integrity is enforced cryptographically by the upstream crate (file-size check plus BLAKE2b-512 hash against pinned constants), but no host-side filesystem hygiene is enforced by the application or required by the runbook. A hostile validator host or a misconfigured deployment can leave the parameter files world-readable, group-writable, or symlinked elsewhere. A privileged attacker on the host can replace the files between startup invocations. The cryptographic hash check catches direct content tampering at startup, but the chain verifies only once, so runtime swaps go undetected.

We classify this issue as minor since content integrity is already enforced cryptographically by the upstream library; the hardening measures are belt-and-suspenders host-side controls.

Recommendation

We recommend the following implementations:

1. Documenting and enforcing file mode 0600
2. Configure directory mode 0700 owned by a dedicated service account
3. Refusing to follow symlinks via `OpenOptions::custom_flags(O_NOFOLLOW)`
4. Running under `systemd` with `StateDirectory=hyved, ReadOnlyPaths=/var/lib/hyved/masp-params, ProtectSystem=strict, and NoNewPrivileges=true`
5. Configuring file integrity monitoring (AIDE, Tripwire, or audited) on the parameters directory.

Status: Resolved

39. No denom allow-list on shield, unshield, or shielded-transfer paths allows accepting any bank denom into the MASP pool

Severity: Minor

In `precompiles/common/masp_tx_helpers.go:77-102`, the `ValidateMaspShieldCall` function validates the `MinShieldAmount` and `MaxShieldAmount` parameters, as well as the sender's bank balance.

The issue is that it does not check `msg.Denom` against any registered allow-list. The `Shield` function in `x/masp/keeper/keeper.go:199-244` accepts any denom the bank module recognizes and writes it under `ShieldedBalancePrefix` and `ShieldedUserBalancePrefix`. The `Params` struct in `x/masp/types/params.go` does not declare an `AllowedDenoms` field.

A malicious or careless integrator can shield denoms that the operator never intended to support. For example, IBC tokens with non-standard decimals, governance tokens with vote-weight side effects, tokens with transfer hooks, or denoms that overlap with internal accounting (e.g., `ahyve` used in `BeginBlocker.AdvanceEpoch` for staking math).

Recommendation

We recommend adding `AllowedDenoms []string` to `x/masp/types/params.go` and rejecting `msg.Denom` in `ValidateMaspShieldCall`, `ValidateMaspUnshieldCall`, and the corresponding ante decorator rules if it is not in the allow-list.

Additionally, the allow-list should be configurable via governance.

Status: Resolved

40. `try_decrypt_note` is an explicit stub returning `None`, preventing wallet syncs

Severity: Minor

In `hyve-masp/src/notes.rs:88-105`, the `try_decrypt_note` function has an in-source comment “STUB – NOT YET IMPLEMENTED” and always returns `None`. The function signature accepts `(ciphertext, viewing_key, _epoch)` and discards both `ciphertext` and `viewing_key` after the `let _ = (ciphertext, viewing_key)` line. The intended decryption of the Sapling note is absent.

Consequently, a recipient cannot scan the chain for incoming shielded notes addressed to their viewing key. Every legitimate `shieldedTransfer` becomes a one-way write: the commitment is appended to the tree, but no party can recover the note plaintext.

Recommendation

We recommend implementing Sapling note decryption following `masp_primitives::note_encryption::try_sapling_compact_note_decryption`: deriving the per-output `epk` · `ivk` shared secret, deriving the symmetric key via Blake2b, decrypting the ciphertext, and returning the parsed `Note`.

Status: Resolved

41. Synthetic simplified-unshield nullifier collides for same-block, same-recipient, same-amount withdrawals

Severity: Minor

In `x/masp/keeper/keeper.go:408-413`, the `unshieldSimplified` flow derives the synthetic pseudo-nullifier as `sha256("simplified|<recipient>|<denom>|<amount>|<height>")`.

This is problematic because two simplified-unshield transactions with identical `(recipient, denom, amount, blockHeight)` tuples produce the same hash. The

first transaction succeeds in calling `RevealNullifier` and writes the entry under `NullifierStoreKey`. The second transaction fails at `RevealNullifier` because the entry already exists.

Consequently, a user who legitimately submits two simplified-unshield transactions in the same block sees the second one revert silently. The user is also forced to reveal the relationship between the two transactions by the very collision. The defect is also a denial-of-service lever: a relayer or front-running observer can pre-empt a victim's simplified-unshield by submitting an identical one before them.

Recommendation

We recommend including a per-transaction unique salt in the hash input, the Cosmos transaction hash, an incrementing per-account nonce, or a combination of (`BlockHeight`, `TxIndex`, `MsgIndex`).

Status: Resolved

42. FFI JSON wire format carries no version byte, so silent schema drift between Rust and Go corrupts verification results

Severity: Minor

In `hyve-masp/src/ffi.rs:303-329`, the `hyve_masp_verify_tx` export returns `serde_json::to_vec(&result)` for the `VerifyResult` struct. The Go side declares the matching struct in `x/masp/ffi/masp_bridge.go:20-26` and decodes via `json.Unmarshal`.

However, there is no version field, no length-prefixed framing, no schema check, and no negative test for unknown fields. Either side can be modified independently, and the other side will silently absorb the change as zero-valued defaults for missing fields.

For example, if a future Rust change drops or renames the `value_balance` field, the Go side will decode `ValueBalance == 0` for every bundle, and the `result.ValueBalance != 0` condition in `precompiles/common/masp_tx_helpers.go:201` will pass incorrectly, accepting any MASP transaction as balanced.

Recommendation

We recommend defining a length-prefixed binary wire format with a leading u32 version byte, encoding (`version`, `payload_len`, `payload`) on the Rust side, and rejecting any unsupported version on the Go side.

As an interim measure, we recommend configuring `serde` to deny unknown fields on the Rust side and configuring Go's `json.Decoder.DisallowUnknownFields` so renamed-field drift surfaces as an error.

Status: Resolved

43. Hardcoded `BranchId::MASP` in `Transaction::read` locks the chain to one `BranchId` without an upgrade path

Severity: Minor

In `hyve-masp/src/proofs.rs:143`, the `verify_masp_transaction` function invokes `Transaction::read(tx_bytes, BranchId::MASP)`. The branch identifier is a hard-coded compile-time constant rather than a value sourced from `Params` or the on-chain epoch state.

In Zcash, `BranchId` rotates during consensus upgrades to invalidate cross-fork transaction replay. However, HYVE inherits the type but not the rotation mechanism. If the upstream Anoma or Zcash ecosystem rotates the `BranchId` for a future MASP version, HYVE cannot follow without a binary redeploy and a coordinated chain upgrade.

Conversely, if HYVE diverges in MASP transaction encoding, the absence of a versioned `BranchId` means in-flight transactions and pre-built proofs cannot be distinguished by the verifier across the upgrade boundary.

Recommendation

We recommend sourcing the `BranchId` from `Params` or from a per-epoch on-chain value by adding a `BranchId uint32` field to `x/masp/types/params.go` and passing it across `cgo` to `verify_masp_transaction`. The current value should be pinned to `BranchId::MASP` at genesis and tunable via governance for future upgrades.

Status: Resolved

44. Orphan FFI exports lack any Go caller, including the safety-critical `hyve_masp_check_invariant`

Severity: Minor

In `hyve-masp/src/ffi.rs`, the FFI exported `hyve_masp_tree_new` in line 337, `hyve_masp_tree_root` in line 354, and `hyve_masp_check_invariant` in line 433 are declared `#[no_mangle] pub unsafe extern "C"`. However, no Go caller exists in `x/masp/ffi/`, `x/masp/keeper/`, or any other package.

This means that the first two are dead code: the `pool_*` family of exports performs the actual tree operations the chain uses. While the third is a silent-fund-drain enabler: an invariant check that is defined but never called means any consistency violation between the pool blob and the keeper's KV state goes undetected. An invariant such as “*tree leaf count equals total*”

note_count” or “every nullifier-set entry has a corresponding revealed-spend record” cannot fire if the chain never calls it.

Recommendation

We recommend wiring `hyve_masp_check_invariant` into `EndBlocker` in `x/masp/keeper/keeper.go` and panicking on a violation, with a Cosmos parameter `EnableInvariantCheck` so operators can disable the check for performance under stress.

We also recommend removing the dead `hyve_masp_tree_new` and `hyve_masp_tree_root` exports unless a near-term caller is planned.

Status: Resolved

45. `keeper.NewKeeper` hardcodes `DefaultParams`, causing genesis-declared parameters to be silently overridden

Severity: Minor

In `x/masp/keeper/keeper.go:35`, the `NewKeeper` function constructs `params := types.DefaultParams()` unconditionally; no constructor argument or KV read `Params` sources from genesis. The `InitGenesis` function in `x/masp/module/module.go:103-117` does not call any `SetParams` keeper method (no such method exists in the codebase), and the keeper has no `Params` getter that reads from KV.

Consequently, every field of `Params` is `types.DefaultParams` returns at compile time. A chain operator declaring `Params` in their genesis file sees no effect on chain behavior, as the values are silently dropped.

Additionally, tuning any parameter requires editing source code and rebuilding the binary, which is impractical for production validators and inconsistent with Cosmos SDK conventions.

Recommendation

We recommend adding `SetParams(ctx sdk.Context, params types.Params)` error to the keeper, writing to a `ParamsStoreKey` declared in `x/masp/types/keys.go`, updating `Params` to read from KV and fall back to `types.DefaultParams` only when the key is absent. Lastly, call the `keeper.SetParams(ctx, gs.Params)` from `InitGenesis`.

We also recommend adding a `MsgUpdateParams` handler gated by the governance authority for runtime tuning.

Status: Resolved

46. Caller-supplied `NullifierSeed` is silently overwritten by the keeper

Severity: Minor

In `precompiles/darkpool/tx.go:65`, the `submitOrder` precompile accepts a `nullifierHash` argument from `calldata` and assigns it to `order.NullifierSeed` before passing the order to the keeper. This causes the keeper in `x/darkpool/keeper/order.go:40` to immediately overwrite the field with `sha256(order.Commitment || order.Sender)`, causing the caller-supplied value to have no effect on stored state.

Consequently, a Solidity integrator who computes a nullifier off-chain finds their value silently replaced by the chain-derived one. Downstream auditors must trace the keeper logic to learn the actual nullifier shape, since there is no event, error, or log indicating that the value was discarded. The ABI surface advertises a `nullifierHash` parameter that is functionally inert.

Recommendation

We recommend either removing `nullifierHash` from the precompile's ABI signature and the `Order` struct so the API contract matches the chain-derived semantics, or honoring the caller-supplied value and removing the keeper-side overwrite.

Status: Resolved

47. `CountOpenOrdersByUser` iterates the entire open-order set on every call instead of maintaining a per-user counter

Severity: Minor

In `x/darkpool/keeper/order.go:159`, the `CountOpenOrdersByUser` function returns `uint64(len(orders))` where `orders` is constructed by iterating every entry in the open-order namespace and filtering by `Sender`. The function is invoked on every `submitOrder` call via the `MaxOpenOrdersPerUser` enforcement in line 33, so its cost grows linearly with the total number of open orders across all users rather than with the per-user open count.

This is problematic because order submission gas and validator wall-clock cost scale with the size of the global open-order set. Combined with the enforcement of `MaxOpenOrdersPerUser`, this results in a denial-of-service amplifier. An attacker with capital to maintain many concurrent open orders increases the cost of every other user's order submission.

Recommendation

We recommend maintaining a per-user open-order counter under a dedicated KV namespace (e.g., `OpenOrderCountByUserPrefix(addr) → uint64`). The counter should be

incremented when `SetOrder` is called with `Status == OrderStatusOpen` and decremented on transitions to `Filled`, `Cancelled`, or `Expired`. With this, the iterating implementation of `CountOpenOrdersByUser` should be replaced with a single KV read.

Status: Resolved

48. `hyve_masp_key_bundle_from_seed` does not validate seed entropy before deriving the spending key

Severity: Minor

In `hyve-masp/src/ffi.rs:157`, the `hyve_masp_key_bundle_from_seed` export copies a 32-byte seed from the caller via `slice::from_raw_parts(seed_ptr, 32)` and passes it directly to `keys::key_bundle_from_seed(&seed)`.

However, there is no check that the seed contains sufficient entropy: whether it is an all-zero seed, an all-0xff seed, a counter-derived seed, or any other obviously-degenerate input is accepted and produces a deterministic key bundle whose spending key inherits the predictability of the seed.

Consequently, a wallet or integrator that constructs the seed from a weak source (e.g., an unseeded RNG, a placeholder constant left in development code, a low-entropy entropy pool early in process startup) receives a MASP spending key that an attacker can recover by guessing the seed.

Recommendation

We recommend rejecting obviously-degenerate seeds at the FFI entry. For example, all-zero, all-0xff, fewer than 16 distinct bytes, or fewer than 64 bits of variation across the 32-byte seed. The function should also return `MASP_ERR_INVALID_PARAMS` on rejection.

Status: Resolved

49. `verify_masp_transaction` accepts masp transactions with transparent legs, even though the chain integration ignores them

Severity: Minor

In `hyve-masp/src/proofs.rs:143`, the `verify_masp_transaction` function parses input bytes via `Transaction::read(tx_bytes, BranchId::MASP)`. The Sapling/MASP transaction format supports transparent inputs and outputs; `Transaction::read` accepts either present or absent inputs and outputs. The HYVE chain

integration on the Go side reads only the shielded portions of the result and never reads or enforces anything about the transparent legs.

The issue is that the `sighash` uses the `tx.txid` shortcut in lines 148–152, which calls `hash_transparent(None)`, which is correct only when there are no transparent legs. When transparent legs are present, `tx.txid` includes a non-empty `transparent_digest` derived from the actual `vin` and `vout`. A bundle containing a transparent `vout` encoding (`recipient`, `denom`, `amount`) is parsed, `sighash`-bound, and signed correctly.

However, the chain reads `Recipient`, `Denom`, and `Amount` from the wrapping `MsgUnshield` regardless. The proof's intended destination is silently discarded.

Recommendation

We recommend rejecting MASP transactions with transparent legs at the verifier level until the chain integration is rewritten to use the destination from the parsed transparent `vout`. Once the chain reads from the transparent `vout`, this check should be removed, and the side-channel `MsgUnshield` fields should be dropped.

Status: Resolved

50. `registerMetaAddress` does not implement the `Label` field

Severity: Informational

In `precompiles/stealth/stealth.go:147–151`, the `registerMetaAddressExec` handler constructs a `MsgRegisterMetaAddress` without setting the `Label` field.

However, the SDK's `MsgRegisterMetaAddress` includes a `Label` field, which is stored in `x/stealth/keeper/keeper.go:60`.

Consequently, all EVM registrations will store an empty label, causing EVM-registered meta-addresses to lack the optional human-readable tag that SDK registrations can carry.

Recommendation

We recommend adding a `string label` parameter to the ABI and the precompile handler so that EVM callers can set the label.

Status: Resolved

51. `getMatchHistory` requires a `pair` argument but discards it

Severity: Informational

In `precompiles/darkpool/query.go:122–143`, the `getMatchHistory` function requires two arguments: a `string pair` and a `*big.Int limit`.

However, the `pair` argument in line 127 is type-checked and then discarded. Only the `limit` is forwarded to the keeper via `p.keeper.GetMatchHistory(ctx, uint64(limit.Int64()))`.

Recommendation

We recommend either forwarding the `pair` argument to the keeper so that results are filtered by `pair` or removing the parameter if not required.

Status: Resolved

52. `getAnnouncements` returns hex-encoded ASCII instead of decoded key bytes

Severity: Informational

In `precompiles/stealth/stealth.go:230-233`, the `getAnnouncementsQuery` function converts `a.EphemeralPubKey` to `[]byte` using a direct string-to-byte conversion `[]byte(a.EphemeralPubKey)`.

However, `[]byte(string)` produces the ASCII bytes of the hex characters, not the decoded key bytes. A client expecting a raw 33-byte compressed public key will instead receive 66 ASCII bytes representing the hex encoding.

Recommendation

We recommend replacing `[]byte(a.EphemeralPubKey)` with `common.FromHex(a.EphemeralPubKey)`.

Status: Resolved

53. `shieldedBalance` query is pool-wide, not per-user

Severity: Informational

In `precompiles/masp/masp.go:278-290`, `shieldedBalanceQuery` returns the total pool balance rather than the caller's share, but is exposed under a name that implies per-user balance. Consumers are likely to misuse it and expose themselves to incorrect behavior.

Although we did not find a direct risk of loss of funds, the naming-vs-behavior mismatch is a reliability hazard.

Recommendation

We recommend either renaming the method to `totalShieldedBalance` in the ABI or rewriting it to return the caller's balance via the correct keeper query.

Status: Resolved

54. Sapling nullifiers and shielded balances are duplicated structurally across Rust and Go state

Severity: Informational

In `hyve-masp/src/pool.rs:17-28`, the `ShieldedPool` blob carries empty `nullifier_set` and `shielded_balances` containers that round-trip on every FFI call. The authoritative copies live on the Go side: nullifiers under `NullifierPrefix` (0x02), shielded balances under `ShieldedBalancePrefix` (0x04), and per-user shielded balances under `ShieldedUserBalancePrefix` (0x09), and all of them are declared in `x/masp/types/keys.go`. The Rust-side containers are dormant duplicates with no read path.

This is problematic because a future refactor that reads from the Rust-side containers (e.g., to wire the orphan `hyve_masp_check_invariant` export) will silently receive empty data. The invariant check returns “no inconsistency” because the structures it inspects are unpopulated, which is the worst-case outcome of an invariant check (i.e., false negatives that mask real divergence).

Additionally, this duplication contributes to FFI wire-format bloat, since every cgo round-trip serializes empty containers.

Recommendation

We recommend choosing a single authority, either from the Go side or the Rust side.

Status: Resolved

55. `ShieldedPool` uses `HashSet` and `HashMap` with the default `RandomState` hasher, producing non-deterministic serialization

Severity: Informational

In `hyve-masp/src/pool.rs:17-28`, the `ShieldedPool` struct declares `nullifier_set: HashSet<[u8; 32]>` and `shielded_balances: HashMap<String, u128>`. Both types use `std::collections::hash_map::RandomState` by default, which means iteration order varies per process and across deterministic re-runs. Any serialization that depends on iteration order produces a different byte sequence on different validators or after a restart.

With that said, the defect is dormant in the current production codepath. As the Go keeper only mutates `commitment_tree` and `note_count` on the Rust side, the two map fields are

never populated by the bridge. The defect activates the moment any future code wires `ffi.PoolRevealNullifier`, `shield_deposit/shield_withdraw`, or makes `hyve_masp_check_invariant` meaningful. At that point, validators producing different serialized pool blobs diverge from consensus, causing the chain to halt.

Recommendation

We recommend replacing `HashSet` with `BTreeSet<[u8; 32]>` and `HashMap` with `BTreeMap<String, u128>`.

Status: Resolved

Appendix

1. Command to compute test coverage

To compute test coverage accurately without including code generated by [protoc-gen-gogo](#) and [protoc-gen-grpc-gateway](#), unrelated file extensions (e.g., `.pb.go`, `.pb.gw.go`) need to be excluded.

The following script is executed to compute the test coverage:

```
#!/usr/bin/env bash
# Coverage report for custom modules (excludes protobuf-generated files).
# Usage: ./scripts/coverage-report.sh [module ...]
# Default: x/settlement x/stealth x/darkpool x/masp

set -euo pipefail

MODULES=("${@:-x/settlement x/stealth x/darkpool x/masp}")
# Expand default into array when no args provided
if [ "$#" -eq 0 ]; then
    MODULES=(x/settlement x/stealth x/darkpool x/masp)
fi

ROOT="$(cd "$(dirname "$0")/.." && pwd)"
cd "$ROOT"

printf "\n  %-28s %8s   %s\n" "MODULE" "COVERAGE" "STATEMENTS"
printf "  %s\n" "-----"

for mod in "${MODULES[@]}; do
    tmp=$(mktemp)
    if go test -coverprofile="$tmp" "./${mod}/..." > /dev/null 2>&1; then
        # Strip protobuf / gRPC gateway / pulsar / stub generated files
        sed -i '' '/\.\pb\.\go/d' "$tmp"
        sed -i '' '/\.\pb\.\gw/d' "$tmp"
        sed -i '' '/pulsar\.\go/d' "$tmp"
        sed -i '' '/proto_stubs\.\go/d' "$tmp"

        total=0; covered=0
        while IFS= read -r line; do
            [[ "$line" == mode:* ]] && continue
            s=$(echo "$line" | awk '{print $(NF-1)}')
            c=$(echo "$line" | awk '{print $NF}')
            total=$((total + s))
            [ "$c" -gt 0 ] && covered=$((covered + s))
        done < "$tmp"

        if [ "$total" -gt 0 ]; then
            pct=$(awk "BEGIN {printf \"%.1f\\", ($covered/$total)*100}")
```

```
        printf "  %-28s %7s%%    (%d/%d)\n" "$mod" "$pct" "$covered" "$total"
    else
        printf "  %-28s %8s    %s\n" "$mod" "n/a" "(no statements)"
    fi
else
    printf "  %-28s %8s    %s\n" "$mod" "FAIL" "(tests failed)"
fi
rm -f "$tmp"
done

printf "\n"
```